

并行与分布式计算

-- 进程

线程简介

基本概念

我们在物理处理器之上用软件构建**虚拟处理器**:

处理器: 提供一组指令以及自动执行一系列指令的能力。

线程: 一个最小软件处理器, 可以在其上下文中执行一系列指令。保存线程上下文意味着停止当前执行并保存在稍后阶段继续执行所需的所有数据。

进程: 一个软件处理器, 可以在其上下文中执行一个或多个线程。执行线程意味着在该线程的上下文中执行一系列指令。

上下文切换

上下文

- **处理器上下文**：存储在处理器寄存器中的用于执行一系列指令（例如堆栈指针、寻址寄存器、程序计数器）的最小值集合。

上下文切换

上下文

- ▶ **处理器上下文**：存储在处理器寄存器中的用于执行一系列指令（例如堆栈指针、寻址寄存器、程序计数器）的最小值集合。
- ▶ **线程上下文**：存储在寄存器和内存中的值的最小集合，用于执行一系列指令（即处理器上下文、状态）。

上下文切换

上下文

- ▶ **处理器上下文**：存储在处理器寄存器中的用于执行一系列指令（例如堆栈指针、寻址寄存器、程序计数器）的最小值集合。
- ▶ **线程上下文**：存储在寄存器和内存中的值的最小集合，用于执行一系列指令（即处理器上下文、状态）。
- ▶ **进程上下文**：存储在寄存器和内存中的值的最小集合，用于线程的执行（即线程上下文，但现在至少也包括 MMU 寄存器值）。

上写文切换

观察

1. 线程共享相同的地址空间。线程上下文切换可以完全独立于操作系统进行。
2. 进程切换通常（在某种程度上）更昂贵，因为它涉及让操作系统进入循环，即陷入内核。
3. 创建和销毁线程比创建和销毁进程便宜得多。

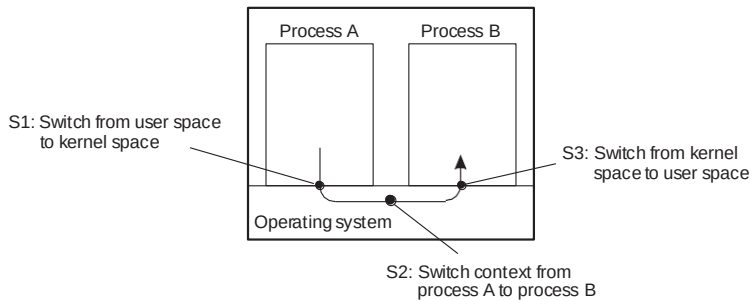
为什么使用线程

部分原因

- ▶ **避免不必要的阻塞：**单线程进程在执行 I/O 时会阻塞；在多线程进程中，操作系统可以将 CPU 切换到该进程中的另一个线程。
- ▶ **利用并行性：**可以安排多线程进程中的线程在多台处理器或多核处理器上并行运行。
- ▶ **避免进程切换：**不要将大型应用程序构造为进程集合，而是通过多个线程。

避免进程切换

避免昂贵的上下文切换



Trade-offs

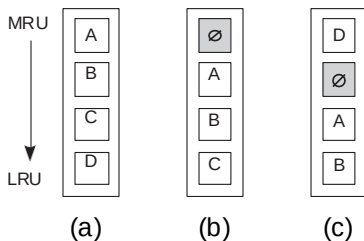
- ▶ 线程使用相同的地址空间：更容易出错
- ▶ 操作系统/硬件不支持保护使用彼此内存的线程
- ▶ 线程上下文切换可能比进程上下文切换更快

上下文切换开销

考虑一个简单的时钟中断处理程序

- ▶ 直接成本：处理程序的实际切换和执行代码
- ▶ 间接成本：其他成本，特别是由缓存混乱引起的成本

间接成本如何产生的？



(a) before the context switch

(b) after the context switch

(c) after accessing block D.

线程与操作系统

主要问题

操作系统内核是否应该提供线程，或者是否应该将其作为用户级包来实现？

用户空间解决方案

- 所有操作均可在**单一进程**内完成⇒ 实施起来可以非常高效。
- 内核提供的所有服务都是代表线程所在进程完成的⇒ 如果内核决定阻塞某个线程，则整个进程将被阻塞。
- 当有大量外部事件时，就会使用线程：线程根据每个事件阻塞⇒ 如果内核无法区分线程，它如何支持向它们发出信号事件？

Threads and operating systems

内核实施方案

整个想法是让内核包含线程包的实现。这意味着所有操作都以系统调用的形式返回：

- ▶ 阻塞线程的操作不再是问题：内核会在同一进程内调度另一个可用线程。
- ▶ 处理外部事件很简单：内核（捕获所有事件）会调度与事件关联的线程。
- ▶ 问题是（或曾经是）效率损失，因为每个线程操作都需要内核陷阱。

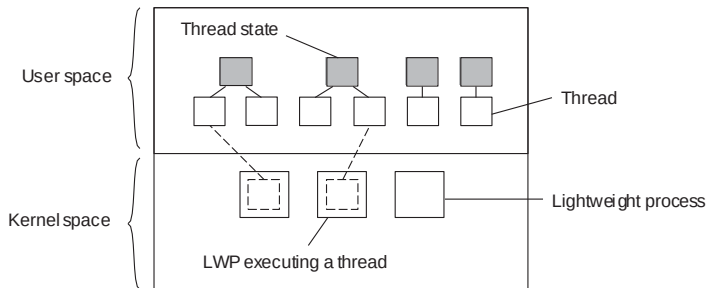
但是

尝试将用户级和内核级线程混合成一个概念，然而，性能提升并没有抵消增加的复杂性。

轻量级进程

基本思想

引入一种两级线程方法：可以执行用户级线程的轻量级进程。



轻量级进程 基本操作

- 用户级线程执行系统调用 $\Rightarrow$ 执行该线程的 LWP 被阻塞。该线程仍然与 LWP 绑定。

轻量级进程 基本操作

- 用户级线程执行系统调用 $\Rightarrow$ 执行该线程的 LWP 被阻塞。该线程仍然与 LWP 绑定。
- 内核可以调度另一个绑定了可运行线程的 LWP。注意：此线程可以切换到当前位于用户空间的任何其他可运行线程。

轻量级进程 基本操作

- 用户级线程执行系统调用⇒执行该线程的 LWP 被阻塞。该线程仍然与 LWP 绑定。
- 内核可以调度另一个绑定了可运行线程的 LWP。注意：此线程可以切换到当前位于用户空间的任何其他可运行线程。
- 一个线程调用阻塞的用户级操作⇒进行上下文切换到可运行线程，（然后绑定到同一个 LWP）。

轻量级进程 基本操作

- 用户级线程执行系统调用⇒执行该线程的 LWP 被阻塞。该线程仍然与 LWP 绑定。
- 内核可以调度另一个绑定了可运行线程的 LWP。注意：此线程可以切换到当前位于用户空间的任何其他可运行线程。
- 一个线程调用阻塞的用户级操作⇒进行上下文切换到可运行线程，（然后绑定到同一个 LWP）。
- 当没有线程需要调度时，LWP 可能保持空闲状态，甚至可能被内核删除（销毁）。

在客户端使用线程

多线程 Web 客户端

隐藏网络延迟：

- ▶ Web 浏览器扫描传入的 HTML 页面，发现需要[获取更多文件](#)。
- ▶ 每个文件由单独的线程获取，每个线程执行（阻塞）HTTP 请求。
- ▶ 文件进入后，浏览器会显示它们。

对其他机器的多个请求-响应调用（RPC）

- ▶ 客户端同时进行多个调用，每个调用由不同的线程执行。
- ▶ 然后等待，直到所有结果都返回。
- ▶ 注意：如果调用不同的服务器，我们可能会[线性加速](#)。

多线程客户端：有帮助吗？

线程级并行：TLP

c_i : i 个线程同时执行的时间比例。

$$TLP = \frac{\sum_{i=1}^N i \cdot c_i}{1 - c_0}$$

其中 N 是可以同时执行的最大线程数。

多线程客户端：有帮助吗？

线程级并行：TLP

c_i : i 个线程同时执行的时间比例。

$$TLP = \frac{\sum_{i=1}^N i \cdot c_i}{1 - c_0}$$

其中 N 是可以同时执行的最大线程数。

实际测量

典型的 Web 浏览器的 TLP 值在 1.5 到 2.5 之间 $\Rightarrow$ 线程主要用于逻辑组织浏览器。

在服务器端使用线程

提高性能

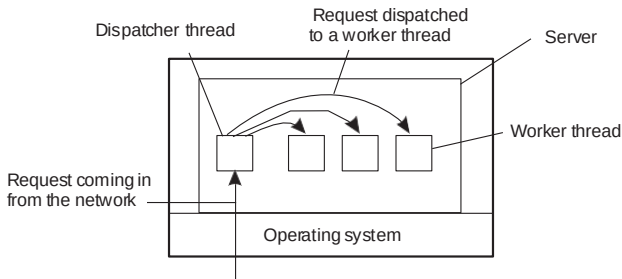
- ▶ 启动一个线程比启动一个新进程更便宜。
- ▶ 单线程服务器无法简单地扩展到多处理器系统。
- ▶ 与客户端一样：在回复上一个请求的同时对下一个请求做出反应，从而隐藏网络延迟。

更好的结构

- ▶ 大多数服务器都有很高的 I/O 需求。使用简单、
- ▶ 易于理解的阻塞调用可以简化整体结构。
- ▶ 由于控制流简化，多线程程序往往更小、更易于理解。

多线程流行的原因：组织

Dispatcher/worker 模型



概览

Model	Characteristics
Multithreading	Parallelism, blocking system calls
Single-threaded process	No parallelism, blocking system calls
Finite-state machine	Parallelism, nonblocking system calls

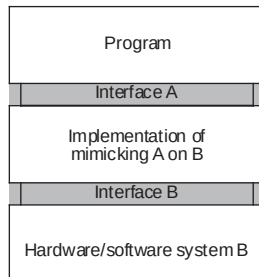
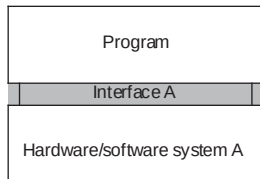
虚拟化

观察

虚拟化 很重要:

- ▶ 硬件比软件变化快
- ▶ 易于移植和代码迁移
- ▶ 隔离故障或受攻击的组件

原理：模仿接口



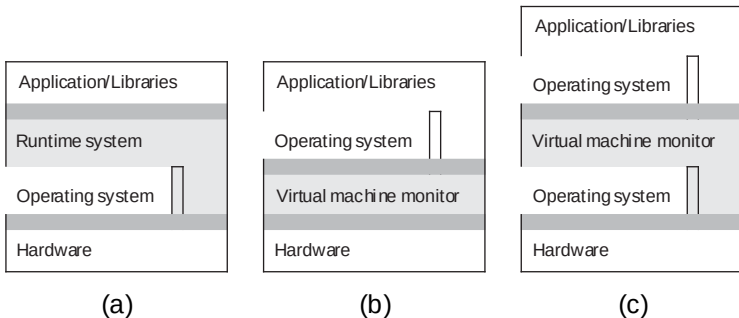
模仿接口

三种不同级别的四种接口

1. **指令集架构**：机器指令集，有两个子集：
 - ▶ 特权指令：仅允许操作系统执行。
 - ▶ 通用指令：可由任何程序执行。
2. 操作系统提供的**系统调用**。
3. **库调用**，称为**应用程序编程接口 (API)**

虚拟化方式

(a) Process VM, (b) Native VMM, (c) Hosted VMM

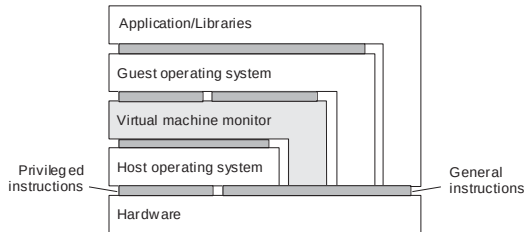


区别

- (a) 独立的指令集，解释器/模拟器，在操作系统上运行。
- (b) 低级指令，以及最基本的最小操作系统
- (c) Low-level instructions, but delegating most work to a full-fledged

深入了解虚拟机：性能

完善组织



- **Privileged instruction:** 特权指令：当且仅当在用户模式下执行时，它会导致操作系统陷入陷阱
- **Nonprivileged instruction** 非特权指令：其余

Special instructions 特殊指令

- **控制敏感指令：**可能影响机器的配置（例如，影响重定位寄存器或中断表的配置）。
- **行为敏感指令：**效果部分由上下文决定（例如，**POPF** 设置中断启用标志，但仅限于系统模式）。

虚拟化的条件

必要条件

对于任何传统计算机，如果该计算机的敏感指令集是特权指令集的子集，则可以构建虚拟机监视器。

问题：条件并不总是满足

可能存在用户在用户模式下执行而不会对操作系统造成陷阱的敏感指令。

解决方案

- ▶ 模拟所有指令
- ▶ 包装非特权敏感指令以将控制权转移到 VMM
- ▶ **半虚拟化**：修改客户操作系统，方法是阻止非特权敏感指令，或使其不敏感（即更改上下文）。

虚拟机和云计算

三种类型的云服务

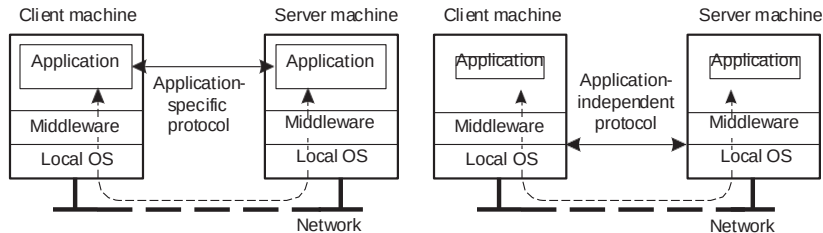
- ▶ **Infrastructure-as-a-Service** 基础设施即服务，涵盖基础设施
- ▶ **Platform-as-a-Service** 平台即服务，涵盖系统级服务
- ▶ **Software-as-a-Service** 软件即服务，包含实际应用程序

IaaS基础设施即服务

云提供商不会出租物理机，而是出租可能与其他客户共享物理机的虚拟机（或 VMM）⇒客户之间几乎完全隔离（尽管可能无法达到性能隔离）。

客户端与服务端交互

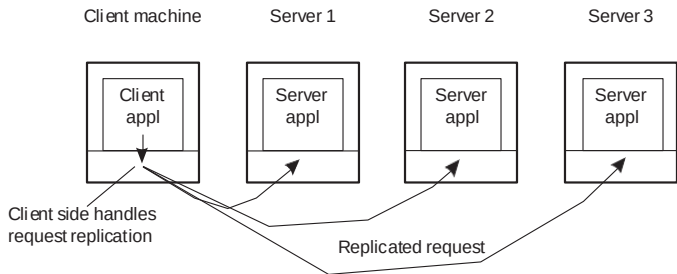
区分应用程序级和中间件级解决方案



客户端软件

通常针对分发透明度进行定制

- ▶ **Access transparency** 访问透明度: RPC 的客户端存根
- ▶ **Location/migration transparency**: 位置/迁移透明度: 让客户端软件跟踪实际位置
- ▶ **Replication transparency** 复制透明度: 客户端存根处理的多个调用:



- ▶ **Failure transparency** 故障透明度: 通常只能放置在客户端（我们试图掩盖服务器和通信故障）。

服务器：一般组织

基本模型

代表一组客户端实现特定服务的进程。它等待来自客户端的传入请求，随后确保请求得到处理，然后等待下一个传入请求。

并发服务器

两种基本类型

- ▶ **Iterative server** 迭代服务器：服务器先处理当前请求，然后再处理下一个请求。
- ▶ **Concurrent server** 并发服务器：使用调度程序，调度程序接收传入的请求，然后将其传递给单独的线程/进程。

观察

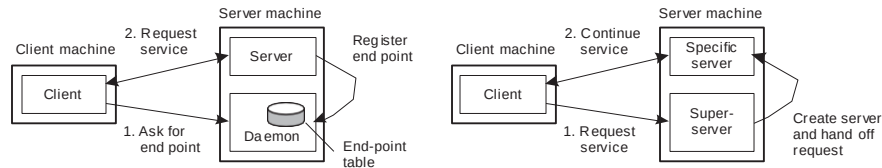
并发服务器是常态：它们可以轻松处理多个请求，特别是在存在阻塞操作（对磁盘或其他服务器）的情况下。

客户端定位

观察：大多数服务都与特定端口绑定

ftp-data	20	File Transfer [Default Data]
ftp	21	File Transfer [Control]
telnet	23	Telnet
smtp	25	Simple Mail Transfer
www	80	Web (HTTP)

动态分配端口



Out-of-band communication

问题

一旦服务器接受了（或者正在接受）服务请求，是否可以中断服务器？

Out-of-band communication

问题

一旦服务器接受了（或者正在接受）服务请求，是否可以中断服务器？

解决方案 1：使用单独的端口来处理紧急数据

- ▶ 服务器有单独的线程/进程来处理紧急消息
- ▶ 紧急消息进来⇒相关请求被搁置
- ▶ 注意：我们要求操作系统支持基于优先级的调度

Out-of-band communication

问题

一旦服务器接受了（或者正在接受）服务请求，是否可以中断服务器？

解决方案 1：使用单独的端口来处理紧急数据

- ▶ 服务器有单独的线程/进程来处理紧急消息
- ▶ 紧急消息进来⇒**相关请求被搁置**
- ▶ **注意：**我们要求操作系统支持基于优先级的调度

解决方案 2：使用传输层的功能

- ▶ 示例：TCP 允许在同一连接中发送紧急消息
- ▶ 可以使用 OS 信令技术捕获紧急消息

服务器和状态

无状态服务器

处理完请求后，切勿保留有关客户端状态的准确信息：

- ▶ 不要记录文件是否已打开（访问后只需再次关闭）
- ▶ 不要承诺使客户端的缓存无效
- ▶ 不要跟踪客户端

服务器和状态

无状态服务器

处理完请求后，切勿保留有关客户端状态的准确信息：

- ▶ 不要记录文件是否已打开（访问后只需再次关闭）
- ▶ 不要承诺使客户端的缓存无效
- ▶ 不要跟踪客户端

后果

- ▶ 客户端和服务端完全独立
- ▶ 由于客户端或服务端崩溃导致的状态不一致减少
- ▶ 可能造成性能损失，例如，服务器无法预测客户端行为

服务器和状态

无状态服务器

处理完请求后，切勿保留有关客户端状态的准确信息：

- ▶ 不要记录文件是否已打开（访问后只需再次关闭）
- ▶ 不要承诺使客户端的缓存无效
- ▶ 不要跟踪客户端

后果

- ▶ 客户端和服务端完全独立
- ▶ 由于客户端或服务端崩溃导致的状态不一致减少
- ▶ 可能造成性能损失，例如，服务器无法预测客户端行为

问题

面向连接的通信是否适合无状态设计？

服务器和状态

状态服务器

跟踪客户端的状态：

- ▶ 记录文件已打开，以便进行预取
- ▶ 了解客户端缓存了哪些数据，并允许客户端保留共享数据的本地副本

服务器和状态

状态服务器

跟踪客户端的状态：

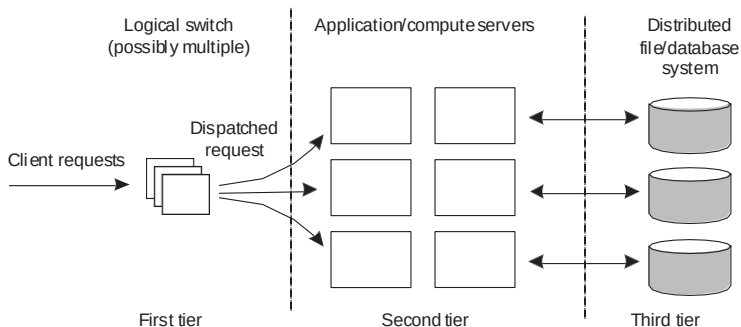
- ▶ 记录文件已打开，以便进行预取
- ▶ 了解客户端缓存了哪些数据，并允许客户端保留共享数据的本地副本

观察

如果允许客户端保留本地副本，有状态服务器的性能可以非常高。事实证明，可靠性通常不是主要问题。

三层结构

一般组织形式



关键要素

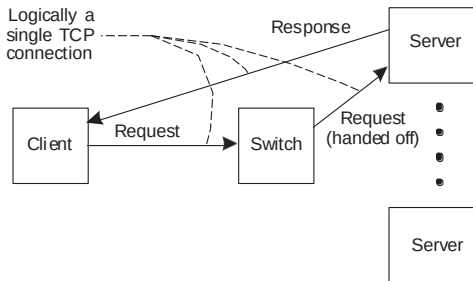
第一层通常负责将请求传递给适当的服务器：请求分派 **request dispatching**

请求处理 Request Handling

观察

让第一层处理来自/到集群的所有通信可能会导致**瓶颈**。

A solution: TCP handoff

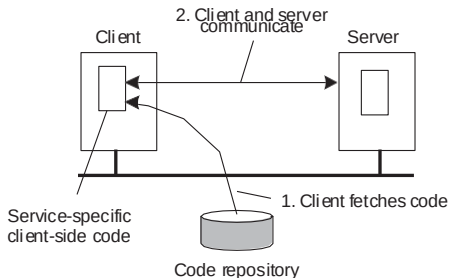


迁移代码的原因

负载分布

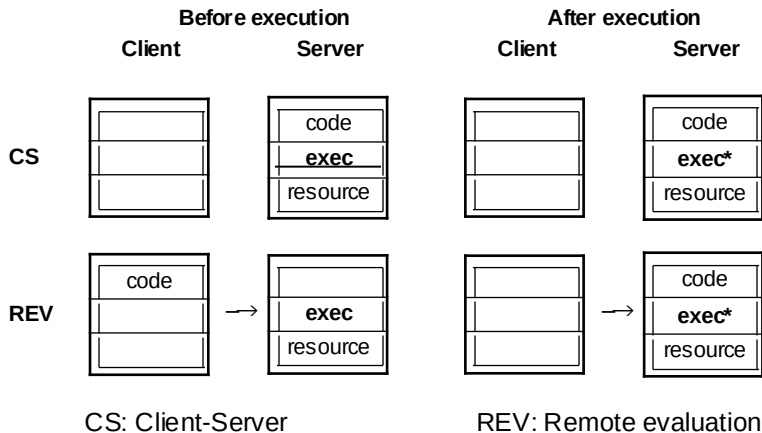
- ▶ 确保数据中心的服务器负载充足（例如，防止能源浪费）
- ▶ 通过确保计算靠近数据所在位置来最大限度地减少通信（想想移动计算）。

灵活性：需要将代码移至客户端

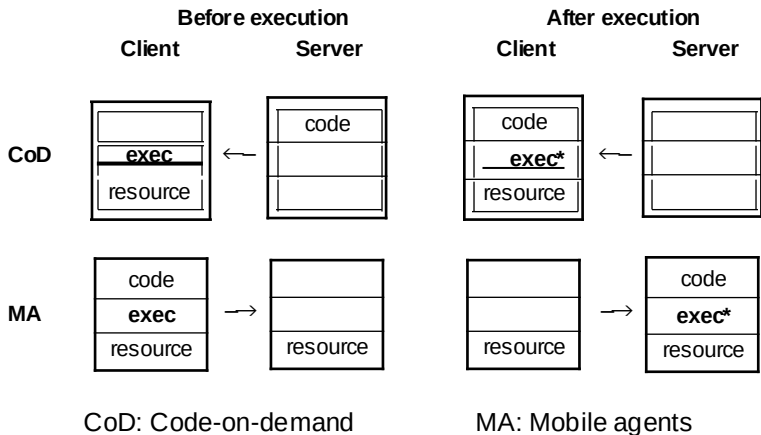


避免预装软件并增加动态配置。

代码迁移模型



代码迁移模型



Strong and weak mobility

对象组件

- ▶ **Code segment**代码段：包含实际代码
- ▶ **Data segment**数据段：包含状态
- ▶ **Execution state**执行状态：包含执行对象代码的线程的上下文

弱移动性：仅移动代码和数据段（并重新启动执行）

- ▶ 相对简单，特别是如果代码是可移植的
- ▶ 区分代码传送（推送）和代码获取（拉取）

强移动性：移动组件，包括执行状态

- ▶ **Migration**迁移：将整个对象从一台机器移动到另一台机器
- ▶ **Cloning**克隆：启动克隆，并将其设置为相同的执行状态。

异构系统中的迁移

问题

- ▶ 目标机器可能不适合执行迁移的代码
- ▶ 进程/线程/处理器上下文的定义高度依赖于本地硬件、操作系统和运行时系统

唯一的解决方案：在不同平台上实现的抽象机

- ▶ 解释型语言，实际上拥有自己的虚拟机
- ▶ 虚拟机监视器

虚拟机迁移

迁移镜像：三种替代方案

1. 将内存页面推送到新机器并重新发送在迁移过程中稍后修改的页面。
2. 停止当前虚拟机；迁移内存，然后启动新虚拟机。
3. 让新虚拟机根据需要拉入新页面：进程立即在新虚拟机上启动并根据需要复制内存页面。

虚拟机迁移性能

问题

完整的迁移实际上可能需要几十秒。我们还需要意识到在迁移过程中，服务将在几秒钟内完全不可用。

虚拟机迁移期间响应时间的测量

